
```
cabinet = [8,4,6,1,2,5,3,7]
newcabinet = []
```

We implement insertion sort by repeatedly calling our `insert_cabinet()` function from [Listing 4-1](#). In order to call it, we'll need to have a file in our “hand,” which we accomplish by popping it out of the unsorted cabinet:

```
to_insert = cabinet.pop(0)
newcabinet = insert_cabinet(newcabinet, to_insert)
```

In this snippet, we used a method called `pop()`. This method removes a list element at a specified index. In this case, we removed the element of `cabinet` at index 0. After we use `pop()`, `cabinet` no longer contains that element, and we store it in the variable `to_insert` so that we can put it into the `newcabinet`.

We'll put all of this together in [Listing 4-2](#), where we define an `insertion_sort()` function that loops through every element of our unsorted cabinet, inserting the elements one by one into `newcabinet`. Finally, at the end, we print out the result, a sorted cabinet called `sortedcabinet`.

```
cabinet = [8,4,6,1,2,5,3,7]
def insertion_sort(cabinet):
    newcabinet = []
    while len(cabinet) > 0:
        to_insert = cabinet.pop(0)
        newcabinet = insert_cabinet(newcabinet, to_insert)
    return(newcabinet)

sortedcabinet = insertion_sort(cabinet)
print(sortedcabinet)
```

[*Listing 4-2: An implementation of insertion sort*](#)

Now that we can do insertion sort, we can sort any list we encounter. We may be tempted to think that this means we have all the sorting knowledge we'll ever need. However, sorting is so fundamental and important that we want to be able to do it in the best possible way. Before we discuss alternatives to insertion sort, let's look at what it means for one algorithm to be better than another and, on an even more basic level, what it means for an algorithm to be good.